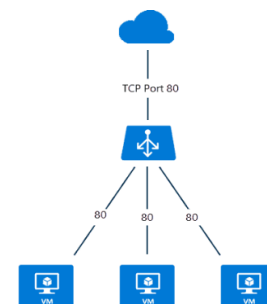


What is Azure Load Balancer?

Azure Load Balancer is a service that distributes incoming traffic to your backend pool. The backend can include **Virtual Machines (VMs)**, **Virtual Machine Scale Sets**, or **containers**. It helps your application stay **highly available** by managing traffic automatically.

It uses a **5-tuple hash algorithm** to decide how to distribute the traffic. The five data points are:

- **Source IP address:** Where the request is coming from
- **Source port:** The port number used by the sender
- **Destination IP address:** Load Balancer's IP address
- **Destination port:** The port the user is trying to reach
- **Protocol type:** Either **TCP** or **UDP**



Why Use Azure Load Balancer?

Azure Load Balancer (especially the **Standard** one) helps scale your apps and ensures **high availability**. It supports both **inbound** and **outbound** traffic and can handle millions of connections.

Key benefits:

- Balance **internal and external** traffic to Azure VMs
- Improve availability across **zones**
- Use **health probes** to check backend resource status
- Allow **port forwarding** to access VMs using public IP and port
- Offers detailed **metrics** via **Azure Monitor**
- Supports multiple **ports** and **IP addresses**

Types of Azure Load Balancer

1. **Public Load Balancer:** Distributes internet traffic to VMs and allows outbound internet access
2. **Internal (Private) Load Balancer:** Balances traffic **within** a virtual network

Lecture 20 – Microsoft Purview and Azure Governance Tools

What is Microsoft Purview?

Microsoft Purview is a set of tools for **data governance**, **risk management**, and **compliance**. It helps you get a complete view of your data across **on-premises**, **multi-cloud**, and **SaaS** environments.

Key features:

- **Automated data discovery**
- **Sensitive data classification**
- **End-to-end data lineage**

Microsoft Purview focuses on two main areas:

1. **Risk and Compliance:** Works with services like **Microsoft Teams**, **OneDrive**, and **Exchange** to:
 - Protect sensitive data
 - Identify risks
 - Help meet compliance rules
2. **Unified Data Governance:**
 - Manage data across **Azure**, **SQL**, **Hive**, **Amazon S3**, etc.
 - Find and classify sensitive data
 - Track how data is used and shared
 - Control access securely at scale

What is Azure Policy?

Azure Policy helps you create and enforce **rules** on your resources to ensure they follow your **organization's standards**. It's used to keep your cloud environment **compliant**.

What are Resource Locks?

Resource Locks prevent accidental changes or deletion of important resources, even by users who have the right permissions. Locks can be placed on a **resource**, **resource group**, or the **entire subscription**.

Types of locks:

- **Delete:** Users can update the resource but **can't delete** it
 - **ReadOnly:** Users can only **view**, but not update or delete the resource
- These locks are **inherited** by all items under them.

Using Virtual Machines (VMs) in Azure

- You can run **single VMs** for small tasks like **development** or **testing**.
- For larger workloads, you can group VMs together for **high availability**, **scalability**, and **redundancy**.
- Azure provides tools like **Availability Sets** and **Virtual Machine Scale Sets** to help you manage VM groups efficiently.

Virtual Machine Scale Sets (VMSS)

- **VM Scale Sets** allow you to create and manage a group of **identical, load-balanced** VMs.
- Normally, if you make multiple VMs, you need to configure each one manually and handle traffic distribution yourself.
- With **VM Scale Sets**, Azure automates: Deployment, Configuration, Scaling, Load balancing
- This helps you run large services like **big data processing**, **compute-heavy applications**, or **container-based workloads** easily and efficiently.

How Applications Handle Fluctuations in Demand

- When user traffic increases, the **scale set automatically adds more VMs** (scales out).
- When traffic decreases, it removes extra VMs to save cost (scales in).
- This **auto-scaling** keeps the app responsive and cost-efficient.

Examples of VM Use

- **Web servers** for hosting websites
- **Database servers** for data storage and queries
- **Development and testing environments**
- **Backup and disaster recovery setups**
- **Running legacy software**

Virtual Machine vs Azure Virtual Desktop

Feature	Virtual Machine	Azure Virtual Desktop
Purpose	Run full OS for apps/services	Provide virtual desktops to users
Users	Usually for backend tasks	Designed for end users to access desktops
Example	Host a web app	Provide Windows desktop access to remote users

Container vs Container App

Feature	Container	Container App
Description	A lightweight package with code, runtime, and dependencies	A managed service to run containers without managing infrastructure
Control	Full control over environment	Azure manages scaling, networking, etc.
Best For	Advanced users needing flexibility	Developers who want fast deployments with less setup

Container vs Virtual Machine

Feature	Container	Virtual Machine
Size	Lightweight	Heavier, full OS
Startup Time	Seconds	Minutes
OS	Shares host OS	Has its own OS
Use Case	Microservices, quick deployments	Full app environments, legacy systems
Isolation	Process-level	Full machine-level isolation

Module 1: Cloud Concepts Overview**Infrastructure as Software**

Cloud computing changes how we think about infrastructure.

Instead of buying and managing **hardware**, you use **infrastructure as software** through the cloud.

Infrastructure as Hardware (Traditional IT)

Using physical hardware means:

- You need **space**, **staff**, and **security**
- You spend a lot of **capital** upfront (called **capital expenditure** or **capex**)
- You have to guess your **maximum usage** in advance

- You face **long wait times** to get and set up hardware

Cloud Computing Deployment Models

These are three ways you can deploy apps using cloud:

1. **Cloud**
 - Everything is deployed and runs **in the cloud**
 - Apps are either **built** in the cloud or **moved** there from old systems
 - You can use basic infrastructure or ready-made services that handle **scaling, security, and management**
2. **Hybrid**
 - Mix of **cloud** and **on-premises** (your local data center)
 - Lets you **connect** your existing setup to the cloud
 - Helps when you want to grow into the cloud but still keep some things on-site
3. **On-Premises (Private Cloud)**
 - Everything runs in your own **data center**
 - Offers **dedicated resources**, but you miss out on cloud benefits
 - Can use tools like **virtualization** to manage resources better

AWS Deployment Models vs Azure

The **deployment models** in **AWS** and **Azure** are very similar:

- **Cloud** = **Public Cloud** in Azure
- **Hybrid** = **Hybrid Cloud** in Azure
- **On-Premises** = **Private Cloud** in Azure

Both platforms let you choose the model that fits your needs best, with flexible options for combining resources.

Similarities Between AWS and Traditional IT

Traditional IT	AWS Equivalent
Firewall / ACL / Admins	AWS Security Groups, IAM, Network ACLs
Routers / Switches	Amazon VPC, Elastic Load Balancing
Servers	Amazon EC2, AMIs
DAS / SAN / NAS / RDBMS	Amazon EBS, EFS, S3, RDS

AWS lets you do almost everything you can do in a traditional data center, but with more flexibility and less effort.

Section 2: Advantages of Cloud Computing

- **Trade Capital Expense for Variable Expense:** Traditional IT needs **capital investment** upfront for data centers and servers (capex). With cloud, you only pay for what you use (variable expense)
- **Benefit from Massive Economies of Scale:** Cloud providers like **AWS** serve millions of customers. They buy hardware in bulk, so you get **lower prices**
- **Stop Guessing Capacity:** You no longer need to guess how much infrastructure you need. **Scale up or down** automatically based on demand
- **Increase Speed and Agility:** You can get **new IT resources** within **minutes**: Developers can experiment and build faster
- **Stop Spending on Maintaining Data Centers:** You don't need to handle servers, power, or cooling
- **Go Global in Minutes:** You can deploy your app in multiple **AWS Regions** worldwide

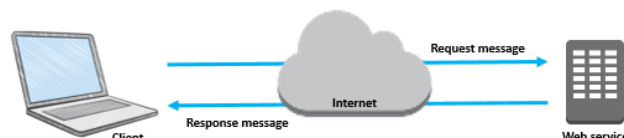
What is AWS?

Amazon Web Services (AWS) is a **secure cloud platform** that gives you access to a wide range of global **cloud-based products**. With AWS, you can use **compute, storage, networking, databases**, and many other **IT services** as you need them.

- You **only pay** for the services you use, and **only while you use them**.
- AWS is **flexible**, so you can choose and combine services like **building blocks** to create your own solutions.

Categories of AWS Services (Names Only)

- **Analytics**
- **Business Applications**
- **Database**
- **Robotics**
- **Machine Learning**



Scenario-Based Question (Use Diagram for Answer)

Question:

You're building a database application. Customers send data to **Amazon EC2**, which batches the data and sends it to **Amazon S3**. Then, you use **Amazon DynamoDB** to create indexes and access the data, all running inside an **Amazon VPC**.

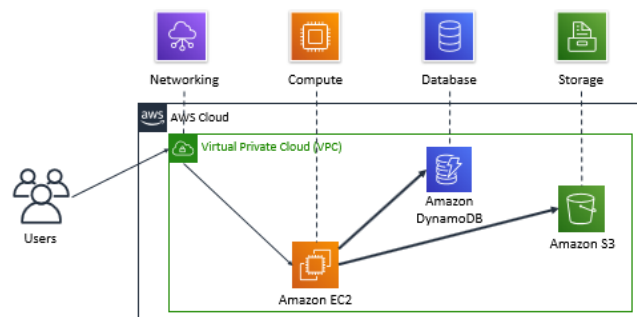
Which AWS services from different categories are used together to build this solution?

Choosing a Service Based on Need

- **Amazon EC2** – You want full control over your computing environment
- **AWS Lambda** – You want to run code without managing servers
- **AWS Elastic Beanstalk** – You want AWS to deploy and manage your web app
- **Amazon Lightsail** – You need a **simple cloud platform** for small apps

Service Examples (Names Only)

- **Compute Services:** Amazon EC2, AWS Lambda, AWS Elastic Beanstalk
- **Database Services:** Amazon Aurora, Amazon DynamoDB, Amazon Redshift
- **Storage Services:** Amazon S3, Amazon S3 Glacier, Amazon EFS
- **Networking and Content Delivery Services:** Amazon VPC, Amazon Route 53, Amazon CloudFront



Three Ways to Interact with AWS

1. **AWS Management Console:** A graphical interface (web UI) for managing AWS services
2. **AWS Command Line Interface (CLI):** Use commands in terminal or scripts to create/manage AWS resources
3. **AWS Software Development Kits (SDKs):** Use **programming languages** like Python, Java, or JavaScript to access AWS and build apps

All of these tools use the same REST-like API behind the scenes.

Sample Exam Question

Q: Why is AWS more economical than traditional data centers for applications with varying compute workloads?

- In traditional data centers, you must **invest in hardware upfront** (capital expense), even if it's not fully used.
- You have to **guess peak workload capacity**, which often leads to **over-provisioning** or **limited performance**.
- With AWS, you use a **pay-as-you-go pricing model**—you only pay for **compute resources** when you actually need them.
- AWS supports **auto-scaling**, so you can **increase or decrease compute power** as your workload changes.
- This avoids the cost of **idle servers** and reduces waste.
- AWS's **global infrastructure** and **economies of scale** allow it to offer **lower prices** than most organizations can achieve on their own.

Module 2: Cloud Economics and Billing

AWS Pricing Model

In AWS, your costs mainly depend on three things:

- **Compute** (how much processing power you use),
- **Storage** (how much data you store), and
- **Outbound data transfer** (how much data you send out from AWS).

Usually, **inbound data transfer** (data coming into AWS) and **data transfer within the same region** is free. But always check the specific service details to confirm.

Outbound data transfer from all services is combined and shown as a charge called **AWS Data Transfer Out** on your bill.

How Do You Pay for AWS?

AWS works like a **utility service**—you only pay for what you use. Here are four main ways you can save:

1. Pay for What You Use

- No need to buy expensive servers or build data centers.
- You only pay for the resources (like compute or storage) that you actually use.

2. Pay Less When You Reserve

- For services like **Amazon EC2**, you can reserve capacity ahead of time and get a discount.

3. Pay Less When You Use More

- As your usage grows, you get **volume discounts**.
- Example: With **Amazon S3**, the more data you store, the lower the price per GB.

- You can also choose cheaper storage options based on how often you access your data (like using **S3 Glacier** for rarely accessed data).

Some Services Have No Additional Cost

These services help manage and deploy resources, and they don't add extra charges:

- **Amazon VPC** – Set up your own isolated network in the AWS Cloud.
- **IAM (Identity and Access Management)** – Manage users and their access to AWS.
- **Elastic Beanstalk** – Deploy and run applications quickly.
- **CloudFormation** – Automatically create and manage AWS resources.
- **Auto Scaling** – Add or remove resources automatically based on demand to control cost and maintain performance.

Section 2: Total Cost of Ownership (TCO)

On-Premises vs Cloud

On-premises infrastructure:

- You own and manage everything (servers, hardware, licenses, etc.).
- Costs are **capital expenses**—paid upfront and fixed.
- Hard to scale up (takes time and money) or scale down (you still pay for everything even if unused).

Cloud infrastructure (like AWS):

- You **rent** what you need from AWS.
- Pay only for what you use (**operational expenses**).
- Scaling up or down is quick and flexible.
- Easier to estimate costs based on your usage.

What is TCO?

Total Cost of Ownership (TCO) means the full cost of running your system, including both **direct** (hardware, software) and **indirect** (power, cooling, staff) expenses.

Why Use TCO?

- To compare the cost of running apps or infrastructure **on-premises** vs **on AWS**.
- To build a **business case** and budget before moving to the cloud.

TCO (Total Cost of Ownership) Considerations

Managing a traditional data center involves several types of costs:

- **Server costs** – Includes hardware, software, and space to keep the servers.
- **Storage costs** – Hardware, maintenance, and physical space.
- **Network costs** – Equipment, admin, and space.
- **IT labor costs** – People needed to manage all systems and resources.

Cloud vs On-Premises (Data Center)

When comparing cloud and on-premises solutions, it's important to understand **all costs involved**:

✓ *Cloud Costs (like AWS):*

- **Easy to estimate** – AWS gives clear pricing based on usage (like RAM, storage, bandwidth).
- **Pay-as-you-go** – You pay only for what you use.
- **Upfront pricing** – Transparent and predictable.

✗ *On-Premises Costs:*

- **Hard to calculate** – Many hidden or indirect costs.
- **Direct costs** – Power, floor space, operations.
- **Indirect costs** – Infrastructure, backup systems, and more.

Also includes:

- **Software licenses** (DBs, middleware)
- **Facility upgrades and maintenance**
- **IT staff for security, admin, etc.**

AWS Pricing Calculator

AWS offers a tool to:

- Estimate **monthly costs**
- Explore **cost-saving opportunities**

- **Model your architecture** before building it
- Find **instance types** and **pricing plans**
- Organize services into named groups for easier tracking

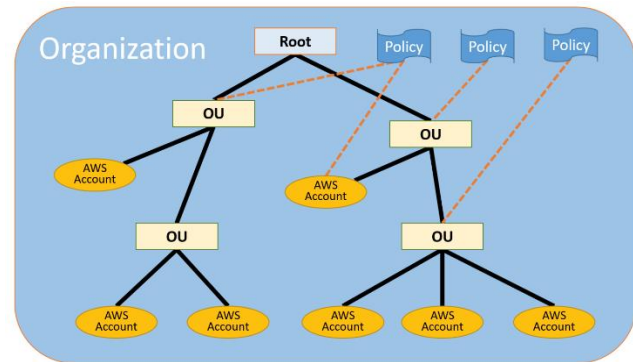
Benefits of Using AWS

Hard Benefits (Measurable):

- Lower spending on **compute, storage, networking, and security**
- Reduce **CapEx** (hardware/software purchases)
- Cut down **operational costs** (backups, disaster recovery)
- Fewer operations staff needed

Soft Benefits (Indirect):

- **Reusable cloud services** speed up development
- Boosts **developer productivity**
- Faster and more **agile** business processes
- Better **customer satisfaction**
- **Global reach** with minimal effort



AWS Organizations: Key Terminology

AWS Organizations helps manage **multiple AWS accounts** under one structure.

- **Root:** Top-level container of all AWS accounts.
- **Organizational Units (OUs):** Like folders to group accounts.
- **Accounts:** Where your AWS resources live.
- **Policies:** Rules applied to OUs or accounts to control access.

 *Policies applied to OUs affect all accounts inside them (like an upside-down tree structure).*

Key Features of AWS Organizations

- Create and manage **Service Control Policies (SCPs)** to restrict or allow actions.
- **Group accounts** and apply policies for consistency.
- **Automate** account creation using **APIs**.
- Use **consolidated billing** for all accounts – helps get **volume discounts** and manage all bills in one place.

How to Set Up AWS Organizations (4 Steps)

1. **Create an organization** – Your current AWS account becomes the primary account.
2. **Create OUs** – Organize member accounts.
3. **Add SCPs** – Restrict or allow actions in accounts.
4. **Test policies** – Log in or use IAM Policy Simulator to verify permissions.

Sample Exam Question

Q: Which AWS service provides infrastructure security optimization recommendations?

AWS Trusted Advisor provides infrastructure security optimization recommendations. Here's a simple breakdown:

- **Security Checks:** It checks for security best practices in your AWS environment (like open ports, IAM permissions, etc.).
- **Performance Optimization:** Suggests ways to improve performance of your AWS resources.
- **Cost Optimization:** Recommends actions to reduce unnecessary spending.
- **Fault Tolerance:** Checks for ways to increase availability and resilience of your setup.
- **Service Limits:** Warns you if you're close to hitting service limits (like EC2 instances or S3 buckets).

Module 3: AWS Global Infrastructure Overview

AWS Regions

- A **Region** is a **geographic location** (like US-East, Asia-Pacific).
- You **control** how and when **data is replicated** across Regions.
- Communication between Regions uses the **AWS global backbone network**.
- Every Region has **full redundancy and network connectivity**.
- A Region includes **two or more Availability Zones**.

Availability Zones (AZs)

- Each Region has multiple AZs.

- An **Availability Zone** is a **separate and isolated part** of the AWS infrastructure.
- AZs are made up of **discrete data centers**.
- Designed for **fault isolation**.
- Connected to each other via **high-speed private networking**.
- You can **choose which AZs** to use.
- AWS recommends **replicating resources across AZs** for **resilience and high availability**.

◆ Points of Presence (PoPs)

- Includes **Amazon CloudFront** (CDN) and **Amazon Route 53** (DNS).
- Used to **distribute content** and **route user requests** with **low latency**.
- Requests are automatically sent to the **nearest edge location** to improve speed.

◆ AWS Infrastructure Features

- **Elasticity**: Automatically **adjusts capacity** based on demand.
- **Scalability**: Grows or shrinks infrastructure as needed.
- **Fault-Tolerance**: Keeps working even when parts fail.
- **Redundancy**: Backup components are built-in.
- **High Availability**: Keeps apps running with **minimal downtime** and **no manual work**.

◆ AWS Foundational Services

- AWS infrastructure is made up of **Regions**, **Availability Zones**, and **Points of Presence**.
- Services run on this infrastructure and include **compute**, **storage**, **networking**, and **databases**.
- Services are **on-demand**, ready in **seconds**, and follow a **pay-as-you-go model**.

◆ AWS Categories of Services: Compute, Networking and Content Delivery, Database, Storage

◆ Storage Service Category

1. **Amazon S3** (Simple Storage Service):
 - **Object storage** for apps, websites, backups, IoT, big data, etc.
 - Offers **scalability**, **availability**, and **security**.
2. **Amazon EBS** (Elastic Block Store):
 - **Block storage** for EC2 instances.
 - Ideal for **databases**, **file systems**, **analytics**, etc.
3. **Amazon EFS** (Elastic File System):
 - **Network file system** that grows/shrinks automatically.
 - Works with **cloud and on-premises** resources.

◆ Compute Service Category

1. **AWS Elastic Beanstalk**:
 - Quickly **deploy and scale web applications**.
 - Supports **Apache**, **IIS**, etc.
2. **AWS Lambda**:
 - **Run code without servers** (serverless).
 - **Pay only for the compute time** used.
3. **Amazon EKS** (Elastic Kubernetes Service):
 - Run and manage **Kubernetes-based containerized applications**.

◆ Database Service Category

1. **Amazon RDS** (Relational Database Service):
 - **Easily manage** relational databases.
 - Supports **MySQL**, **PostgreSQL**, **SQL Server**, etc.
2. **Amazon Aurora**:
 - High-performance **MySQL/PostgreSQL-compatible** database.
 - **Faster** than standard open-source databases.
3. **Amazon Redshift**:
 - Run **analytics** on **petabytes** of data.
 - Integrates with **S3** for large-scale data processing.

◆ Security, Identity, and Compliance Category

1. **AWS IAM** (Identity and Access Management):
 - Manage **users, groups, and permissions** securely.
2. **AWS Organizations**:
 - Apply **service and action restrictions** across multiple AWS accounts.
3. **Amazon Cognito**:
 - Add **user sign-up and login** to web and mobile apps.
4. **AWS Artifact**:
 - Access **security and compliance reports** on demand.
5. **AWS KMS** (Key Management Service):
 - Create/manage **encryption keys** for your data and services.
6. **AWS Shield**:
 - Protects applications against **DDoS attacks** (Distributed Denial of Service).

Question #1:

Under which service category does the IAM service appear?

- IAM (Identity and Access Management) appears under the "**Security, Identity, & Compliance**" category.

Question #2:

Under which service category does the Amazon VPC service appear?

- Amazon VPC (Virtual Private Cloud) appears under the "**Networking & Content Delivery**" category.
- VPC lets you **provision a logically isolated network** in the AWS Cloud.
- You can define your own **IP address range, subnets, route tables, and gateways**.
- It provides **complete control over your virtual networking environment**.

Question #5:

Which of the following services are global instead of Regional?

(Check Amazon EC2, IAM, Lambda, and Route 53)

- ☒ **IAM** – Global service
- ☒ **Route 53** – Global service
- ☒ **Amazon EC2** – Regional service
- ☒ **AWS Lambda** – Regional service

Describe IAM and Route 53:

IAM (Identity and Access Management):

- IAM helps you **securely control access** to AWS services and resources.
- It lets you **create users**, assign **permissions**, and define **policies**.
- IAM is a **global service**, so it doesn't depend on a specific AWS Region.

Route 53:

- Route 53 is a **Domain Name System (DNS) web service**.
- It helps route users to **applications hosted in AWS** or elsewhere.
- It supports **domain registration, DNS routing, and health checking**.
- Route 53 is a **global service**, operating across all Regions.

Describe Regional Services (Amazon EC2 and Lambda):

Amazon EC2 (Elastic Compute Cloud):

- Amazon EC2 provides **virtual servers (instances)** to run your applications.
- It's a **regional service**, so you **launch EC2 instances in specific Regions**.
- Helps you control **compute capacity** and scale easily.

AWS Lambda:

- Lambda lets you **run code without provisioning or managing servers**.
- It is **event-driven** and automatically scales with usage.
- Lambda is a **regional service**, and functions are deployed to **specific Regions**.

AWS Shared Responsibility Model

- **Security and compliance** are shared between **AWS** and the **customer**.
- This model is divided into:
 - **Security of the cloud** – AWS's job
 - **Security in the cloud** – Customer's job

AWS Responsibility: Security *of* the Cloud

AWS is responsible for the global infrastructure:

- **Data center security** (guards, cameras, badge access, etc.)
- **Hardware** like servers and storage
- **Software infrastructure** (host OS, virtualization layer)
- **Network** (routers, switches, firewalls, etc.)
- **Global infrastructure** like **Regions**, **Availability Zones**, and **edge locations**
- AWS provides **third-party audit reports** to show compliance

Customer Responsibility: Security *in* the Cloud

Customers are responsible for:

- **Data encryption** (at rest and in transit)
- **Security groups, firewalls, network configurations**
- **IAM policies, user access, and login credentials**
- **Operating system patches and application updates**
- Deciding:
 - **What data** to store
 - **Which services** to use
 - **Where** to store data
 - **Who can access** the data

Service Types and Security Responsibilities

📖 *Infrastructure as a Service (IaaS) – More customer responsibility*

Example: Amazon EC2

- Customer manages:
 - Guest OS
 - Security patches
 - App software
 - Security groups and firewall rules

📖 *Platform as a Service (PaaS) – Shared responsibility*

Examples: AWS Lambda, Amazon RDS

- AWS manages:
 - Infrastructure
 - OS patches
 - Database updates
- Customer manages:
 - Data
 - User permissions
 - App logic

📖 *Software as a Service (SaaS) – Less customer responsibility*

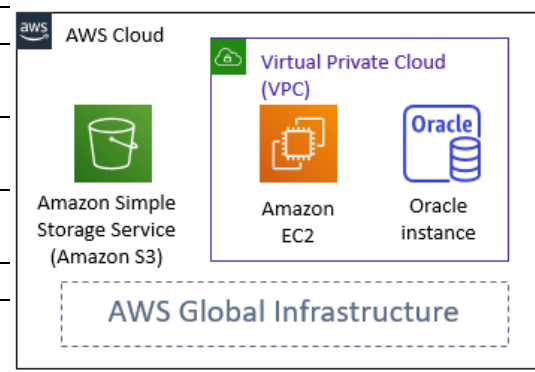
Examples: AWS Trusted Advisor, AWS Shield, Amazon Chime

- AWS manages:
 - Everything under the hood
- Customer uses the service via browser or API
- Example:
 - **Trusted Advisor** gives best practice recommendations
 - **AWS Shield** protects from DDoS attacks
 - **Amazon Chime** is used for online meetings and calls

Activity: Scenario 1 of 2

Consider this deployment. Who is responsible – AWS or the customer?

Component	AWS Responsibility	Customer Responsibility
Amazon S3	Physical infrastructure, storage service	Access control, data encryption, policies
Amazon EC2	Virtualization, hardware	OS updates, app maintenance, security groups
Amazon VPC	Underlying networking platform	VPC setup, routing, network ACLs, firewall configuration
Oracle on EC2	None	DB install, patching, admin, scaling
Oracle on RDS	Backup, patching, scaling, monitoring	Data access, DB schema, user-level security



Activity: Scenario 2 of 2

Consider this deployment. Who is responsible – AWS or the customer?

Component	AWS Responsibility	Customer Responsibility
Amazon S3	Physical infrastructure, storage durability	Data access control, encryption, bucket policies, versioning, permissions
Amazon EC2	Virtualization, physical servers, networking hardware	OS updates, patching, app deployment, securing SSH keys, configuring security groups
Amazon VPC	Underlying network infrastructure	VPC creation, subnet configuration, routing, internet gateway setup, ACLs, firewalls
Internet Gateway	Availability and performance	Attaching it to VPC, configuring route tables
AWS CLI	Secure APIs and endpoints	Use of access keys, securing credentials, enforcing MFA
SSH Access	Infrastructure support for secure protocols	SSH key management, user access control
IAM (optional)	IAM service availability and operation	IAM policy setup, user/group/role definitions, MFA enforcement

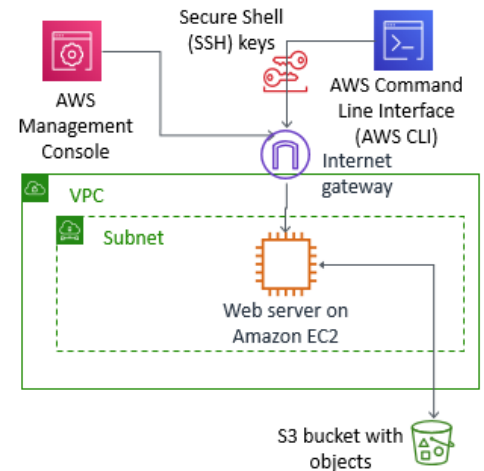
IAM (Identity and Access Management)

IAM lets you manage **who can access what** in your AWS account.

- **Manages access** to services like EC2, S3, RDS, etc.
- IAM supports:
 - **Authentication** (who are you?)
 - **Authorization** (what can you do?)

IAM Components

1. **IAM User**
 - A person or app that accesses AWS using **API keys or passwords**
2. **IAM Group**
 - A collection of users. Easier to manage permissions.
3. **IAM Policy**
 - A document that defines what **users or roles can do**
4. **IAM Role**
 - Grants **temporary access** to AWS resources



Lecture 18 (Before Midterm)

Layered Cloud Architecture Development

1. Infrastructure Cloud

- This includes the **physical components** used to run applications and store data.
- **Virtual servers** run applications.
- **Virtual storage pools** store data in different storage tiers based on **performance and reliability**.
- Uses **virtual abstraction**, so servers and storage are managed as logical (not physical) units.

2. Content Cloud

- Builds on the infrastructure cloud using **metadata** and **indexing services**.
- Manages data independently from applications.

- Turns data into **objects**, so it can be accessed by different applications even if the original app is gone.

3. Information Cloud

- Abstracts the **client from the data**.
- For example, a user in the U.S. can access a video stored in Japan using a mobile phone.
- This layer abstracts **everything from everything**, making global access easy.

Cloud Security Architecture

- Defines how to **design and structure** security for platforms, tools, infrastructure, and software in the cloud.
- Helps configure and secure operations using:
 - **Identity and access management**
 - **Application and data protection**
 - **Compliance and threat visibility**
 - **Security principles in development**
 - **Governance and compliance policies**
 - **Physical infrastructure security**

Importance of Cloud Security

- As more business happens in the cloud, **cybersecurity** becomes very important.
- **Cyber threats** aim to steal or damage data or disrupt digital life.

Common Types of Cyber Attacks

- **Malware** – spyware, ransomware, viruses
- **Phishing**
- **Man-in-the-middle attacks**
- **Denial-of-Service (DoS)**
- **SQL Injection**
- **Zero-day exploits**
- **DNS Tunneling**

Cloud Security Solutions Protect Against:

- **External threats** like ransomware, phishing, DoS, brute force, and data breaches
- **Insider threats** like rogue employees or human errors
- **System vulnerabilities** like flaws in applications, procedures, or security controls

Web API

- **API (Application Programming Interface)** is a set of functions to access features or data of an application.
- A **Web API** can be used over the web using **HTTP**.
- It is built with technologies like **Java** or **ASP.NET**.
- Mainly used in **web development** for **client-side applications**.

Cloud Computing Platforms and Technologies

Amazon Web Services (AWS)

- AWS provides many **IaaS** services, such as:
 - **EC2 (Elastic Compute Cloud)** – customizable **virtual servers** for computing
 - **S3 (Simple Storage Service)** – **object storage** for data of any size
- **EC2** allows users to:
 - Launch virtual servers through the **AWS Console** or **Web API**
 - Save running instances as **images** to create reusable templates
- **S3** stores data in **buckets** as **binary objects**, accessible globally.
- AWS also offers services like **networking**, **DNS**, **databases**, and **caching**.

Microsoft Azure

- Azure is a **cloud operating system and platform**.
- It provides a **scalable runtime** for web and distributed applications.
- Applications are organized into **roles**, which define logic and distribution.
- Azure also offers services like **storage**, **networking**, **caching**, and **content delivery**.